AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science
Faculty of Science and Technology

CSC2210: OBJECT ORIENTED PROGRAMMING 2

Summer 2025-2026

Section: [V]

Group No: 11

Project Report On

Project Name ***[Car Rental Management System]***

Supervised By

Syeda Asrafa Islam

Submitted By:

Name	ID
1. Md Isthiyak Mahmud	**24-57795-2**
2. Jannatul Ferdous Mim	**24-57801-2**

Obtained Marks for CO2 and CO4 (Description given in the following page)

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)

Evaluation Criteria (CO2)	Total =	Evaluation Criteria (CO4)	Total =
Requirement fulfillment		Design & Innovation	
Validation		Teamwork & Ethics	
Verification		Communication	

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.					
Assessment Criteria	**Not Attended/ Incorrect (0)**	**Inadequate (1-2)**	**Average (3)**	**Good (4)**	**Excellent (5)**
Evaluation Criteria	**Evaluation Definition**				
Requirement fulfillment	Fails to demonstrate any understanding of real-life scenario-based project development or functional requirement identification. There is no attempt to depict a project or identify functional requirements accurately.	Demonstrates limited understanding of real-life scenario-based project development and functional requirement identification. The project depicted lacks coherence or relevance to real-life scenarios, and functional requirements are inaccurately identified or insufficiently described.	Presents a basic depiction of a real-life scenario-based project and identifies some functional requirements. However, the project lacks depth or complexity, and some functional requirements may be vaguely defined or missing key details.	Effectively demonstrates a realistic scenario-based project and accurately identifies most functional requirements. The project is well-developed with appropriate complexity, and functional requirements are clearly articulated with relevant details.	Exhibits an exceptional understanding of real-life scenario-based project development and accurately identifies all functional requirements. The project is meticulously developed with thorough attention to detail, reflecting a comprehensive understanding of Object-Oriented Programming project development activities.
Validation	Fails to demonstrate any understanding or implementation of validation forms in their system. There is no attempt to deal with data validation, and validation requirements are completely ignored or incorrectly applied.	Demonstrates limited understanding of validation forms and data validation techniques. While some attempt may be made to implement validation, it is incomplete or poorly executed, leading to inadequate handling of data validation.	Shows a basic understanding of validation forms and data validation techniques. They attempt to implement validation, but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistent handling of data validation.	Effectively demonstrates the use of validation forms and implements data validation techniques. Validation is mostly accurate and comprehensive, ensuring the proper handling of data input and verification in the system.	Exhibits an exceptional understanding and implementation of validation forms and data validation techniques. Validation is meticulously implemented with thorough attention to detail, ensuring robust data validation procedures and contributing to the overall reliability and integrity of the system.
Verification	Fails to demonstrate any attempt to verify the system data or functional requirements. There is no	Demonstrates limited understanding of verification processes and data flow in the system.	Shows a basic understanding of verification processes and attempts to verify system data. However,	Identifies and verifies system data, ensuring proper functional requirements are met. Verification efforts are mostly	Exhibits an exceptional understanding of verification processes and meticulously verifies system

	evidence of understanding or implementation of verification processes, and data flow is not considered.	Verification attempts are incomplete or inaccurate, and there is insufficient consideration given to ensuring data integrity and functionality.	verification efforts may be inconsistent or lack thoroughness, and there may be gaps in ensuring proper functional requirements and data flow.	accurate and thorough, with attention to ensuring data integrity and appropriate data flow within the system.	data. Verification efforts are comprehensive and precise, with a keen focus on ensuring all functional requirements are met and maintaining proper data flow throughout the system.

CO4: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.					
Assessment Criteria	**Not Attended/ Incorrect (0)**	**Inadequate (1-2)**	**Average (3)**	**Good (4)**	**Excellent (5)**
Evaluation Criteria	**Evaluation Definition**				
Requirement fulfillment	Fails to demonstrate any attempt to perform or solve system functional requirements. There is no evidence of addressing real-life scenario-based projects or identifying functional requirements for Object-Oriented Programming project development activities.	Demonstrates limited understanding of system functional requirements and real-life scenario-based projects. Attempts to perform or solve functional requirements may be incomplete or inaccurately applied, and there is insufficient consideration given to proper identification of requirements.	Shows a basic understanding of system functional requirements and attempts to perform or solve them. However, the execution may lack depth or thoroughness, and there may be gaps in addressing all aspects of real-life scenario-based projects or properly identifying functional requirements.	Effectively demonstrates the performance and solution of system functional requirements for real-life scenario-based projects. Requirements are identified with clarity and relevance, and the execution is mostly accurate and comprehensive, reflecting a solid understanding of Object-Oriented Programming project development activities.	Exhibits an exceptional understanding of system functional requirements and adeptly performs and solves them for real-life scenario-based projects. Requirements are meticulously identified and executed with thorough attention to detail, showcasing a comprehensive mastery of Object-Oriented Programming project development activities.
Validation	Fails to demonstrate any understanding or implementation of validation forms or data validation techniques. There is no evidence of addressing validation requirements or dealing with data validation in their system.	Demonstrates limited understanding of validation forms and data validation techniques. While some attempt may be made to implement validation, it is incomplete or poorly executed, leading to	Shows a basic understanding of validation forms and data validation techniques. They attempt to implement validation, but some aspects may be missing or incorrectly implemented, resulting in partial or	Demonstrates the use of validation forms and implements data validation techniques. Validation is mostly accurate and comprehensive, ensuring the proper handling of data input and verification in the system, although	Exhibits an exceptional understanding and implementation of validation forms and data validation techniques. Validation is meticulously implemented with thorough attention to detail, ensuring

		inadequate handling of data validation.	inconsistent handling of data validation.	there may be minor issues.	robust data validation procedures and contributing to the overall reliability and integrity of the system.
Verification	Fails to demonstrate any understanding or attempt to verify system data or functional requirements. There is no evidence of addressing verification requirements or considering data flow in their system solution.	Demonstrates limited understanding of verification processes and data flow in the system. Verification attempts may be incomplete or inaccurately applied, and there is insufficient consideration given to ensuring proper functional requirements and data flow.	Shows a basic understanding of verification processes and attempts to verify system data. However, verification efforts may lack thoroughness or precision, and there may be gaps in ensuring proper functional requirements and data flow within the system.	Effectively identifies and verifies system data, ensuring proper functional requirements are met. Verification efforts are mostly accurate and comprehensive, with attention to ensuring data integrity and appropriate data flow within the system.	Exhibits an exceptional understanding of verification processes and meticulously verifies system data. Verification efforts are comprehensive and precise, with a keen focus on ensuring all functional requirements are met and maintaining proper data flow throughout the system.

Table of Contents: **Page no.**

1. Chapter: 01 (Introduction)---------- 06
2. Chapter: 02 (User Story)---------- 06
3. Chapter: 03a (ER Diagram)---------- 07
4. Chapter: 03b (SQL Queries)---------- 09
5. Chapter: 04 (Screenshots)---------- 11
6. Chapter:05 (Code)---------- 20

Chapter 01

Introduction:
Car Rental Management System is a robust, desktop-based enterprise application developed using C# and the .NET Windows Forms framework, designed to streamline, centralize, and automate the daily operations of a vehicle rental business. Built with a clean, modular architecture, the system integrates a secure user login interface with a comprehensive main dashboard that gives administrators full control over five core operational modules: Customer Management, Fleet (Car) Details, Rental Processing, Driver Information, and Maintenance Tracking. By leveraging Object-Oriented Programming (OOP) principles in C# and executing full CRUD (Create, Read, Update, Delete) operations against a relational database backend, the application efficiently manages active rental transactions, tracks fleet availability, logs driver assignments, and monitors vehicle maintenance costs, effectively replacing manual record-keeping with a secure, scalable digital solution.

Chapter 02

User Story:

User1: Admin:

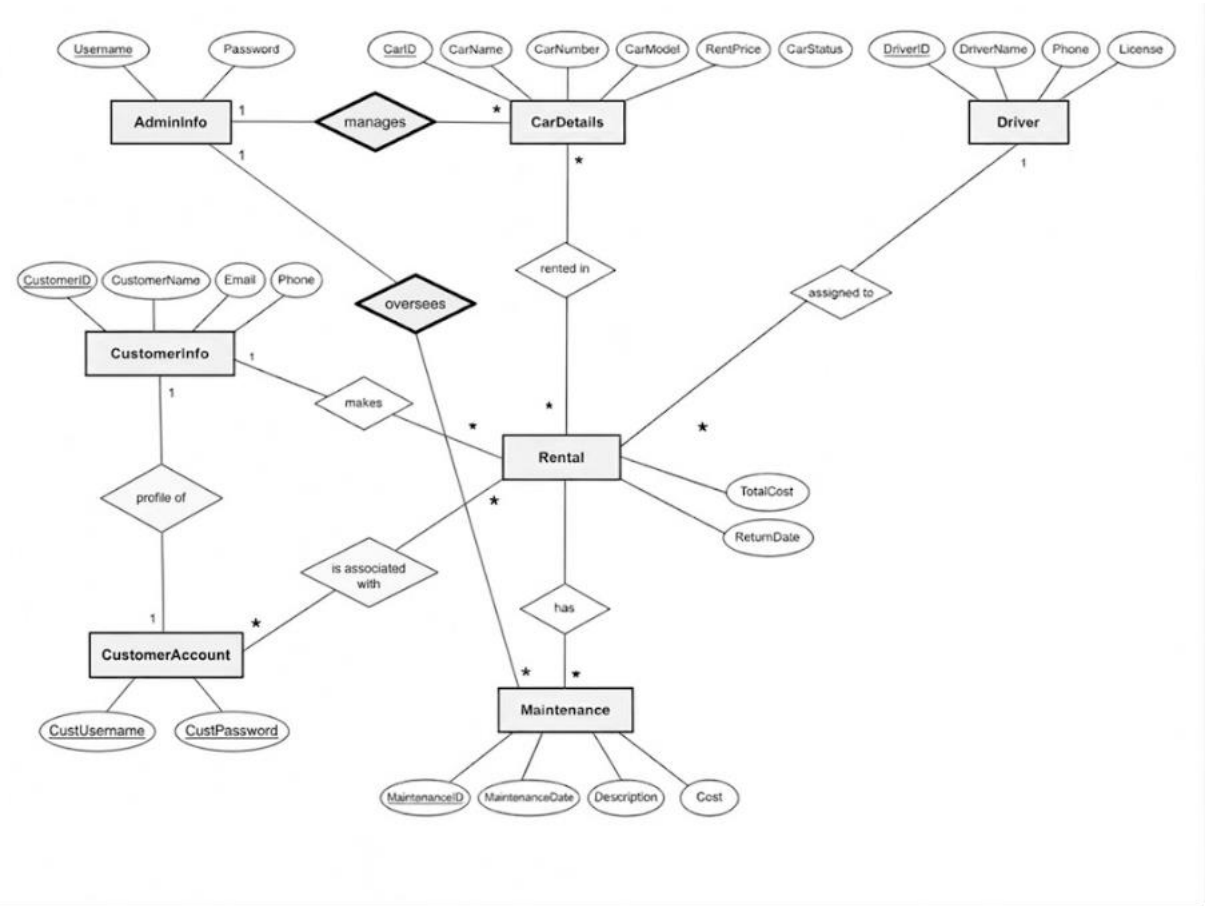
The Admin Page (labeled as the Main Form in my interface) serves as the centralized command center and navigation hub for the entire application. Designed with a dark teal and crimson color scheme, it acts as a secure dashboard that bridges the initial user authentication phase with the system's operational databases. Instead of cluttering the user experience, it features a prominent top navigation menu with five dedicated button pathways: Customer, Car, Rental, Driver, and Maintenance. Clicking any of these buttons triggers event-handling logic in C# to seamlessly open the corresponding sub-form. By placing an abstract, high-tech vehicle management graphic at the center, the page provides the administrator with a clean, uncluttered visual starting point, ensuring they can effortlessly oversee fleet logistics, track active transactions, and pivot between different management modules without losing operational workflow momentum.

User2: Customer (Login & Dashboard Module)
The Customer module extends the Car Rental Management System by allowing registered users to access the system through a secure login process. After entering valid credentials, the customer is redirected to a personalized dashboard designed specifically for rental-related activities.
Unlike the Admin module, the Customer role has limited permissions and is focused only on user-side operations. The Customer Dashboard provides a simple and clear interface that helps users interact with the system without complexity.
The main features available in the Customer Dashboard include:
- **View Available Cars:** Displays all currently available vehicles for rent along with key details such as model, category, rental price, and availability status. This helps customers choose suitable vehicles based on their needs.

• **Rent a Car:** Allows customers to select a vehicle, set rental duration, and submit a rental request. The request is stored in the database and later processed by the admin. This module ensures a proper multi-user system by separating customer access from administrative control. It improves usability, enhances security, and provides a smooth and realistic car rental experience similar to real-world rental platforms.

Chapter 03 (A)

3.1: ER Diagram:



3.2: Normalization:

1. Login Relationship (UserInfo →CustomerInfo)

Relationship: One-to-One (1:1)

- UNF: Username, Password, CustomerID, CustomerName, Gender, Email, Phone
- 1NF: Username, Password, CustomerID, CustomerName, Gender, Email, Phone
- 2NF:
    1. Username, Password
    2. CustomerID, CustomerName, Gender, Email, Phone, Username(FK)

2. Makes Relationship (Customer →Rental)

Relationship: One-to-Many (1:M)

- UNF: CustomerID, CustomerName, Gender, Email, Phone, RentalID, RentDate, ReturnDate, TotalCost, RentalStatus
- 1NF: CustomerID, CustomerName, Gender, Email, Phone, RentalID, RentDate, ReturnDate, TotalCost, RentalStatus
- 2NF:
    1. CustomerID, CustomerName, Gender, Email, Phone
    2. RentalID, RentDate, ReturnDate, TotalCost, RentalStatus, CustomerID(FK)

3. Rented In Relationship (Car →Rental)

Relationship: One-to-Many (1:M)

- UNF: CarID, CarName, CarNumber, CarModel, RentPrice, CarStatus, RentalID, RentDate, ReturnDate, TotalCost, RentalStatus
- 1NF: CarID, CarName, CarNumber, CarModel, RentPrice, CarStatus, RentalID, RentDate, ReturnDate, TotalCost, RentalStatus
- 2NF:
    1. CarID, CarName, CarNumber, CarModel, RentPrice, CarStatus
    2. RentalID, RentDate, ReturnDate, TotalCost, RentalStatus, CarID(FK)

4. Assigned To Relationship (Driver→Rental)

Relationship: One-to-Many (1:M)

- UNF: DriverID, DriverName, Gender, Phone, RentalID, RentDate, ReturnDate, TotalCost, RentalStatus
- 1NF: DriverID, DriverName, Gender, Phone, RentalID, RentDate, ReturnDate, TotalCost, RentalStatus
- 2NF:
    1. DriverID, DriverName, Gender, Phone
    2. RentalID, RentDate, ReturnDate, TotalCost, RentalStatus, DriverID(FK)

5. Has Relationship (Car→Maintenance)

Relationship: One-to-Many (1:M)

- UNF: CarID, CarName, CarNumber, CarModel, RentPrice, CarStatus, MaintenanceID, Service, Cost
- 1NF: CarID, CarName, CarNumber, CarModel, RentPrice, CarStatus, MaintenanceID, Service, Cost
- 2NF:
    1. CarID, CarName, CarNumber, CarModel, RentPrice, CarStatus
    2. MaintenanceID, Service, Cost, CarID(FK)

6. Admin Authentication & Management (AdminInfo)

- Relationship: Independent System Controller (Manages Cars, Drivers, Rentals, and Maintenance)
- UNF / 1NF: AdminUsername, AdminPassword
- 2NF:
    1. AdminUsername, AdminPassword

Finalization (Merged 2NF Entities)

1. UserInfo: Username, Password
2. CustomerInfo: CustomerID, CustomerName, Gender, Email, Phone, Username(FK)
3. CarDetails: CarID, CarName, CarNumber, CarModel, RentPrice, CarStatus
4. Driver: DriverID, DriverName, Gender, Phone

5. Rental: RentalID, RentDate, ReturnDate, TotalCost, RentalStatus, CustomerID(FK), CarID(FK), DriverID(FK)
6. Maintenance: MaintenanceID, Service, Cost, CarID(FK)
7. AdminInfo: AdminUsername, AdminPassword

Final Tables (With Primary Keys & Foreign Keys)

- UserInfo (Username, Password)
- CustomerInfo (CustomerID, CustomerName, Gender, Email, Phone, Username [FK])
- CarDetails (CarID, CarName, CarNumber, CarModel, RentPrice, CarStatus)
- Driver (DriverID, DriverName, Gender, Phone)
- Rental (RentalID, RentDate, ReturnDate, TotalCost, RentalStatus, CustomerID [FK], CarID [FK], DriverID [FK])
- Maintenance (MaintenanceID, Service, Cost, CarID [FK])
- AdminInfo (AdminUsername, AdminPassword)

Chapter 03 (B)

SQL Queries:

Login:

- "SELECT AdminUsername, AdminPassword FROM AdminInfo WHERE AdminUsername=@username AND AdminPassword=@password";
- "SELECT Username, Password FROM UserInfo WHERE Username=@username AND Password=@password";

Customer:

- SELECT * FROM customerinfo;
- "INSERT INTO UserInfo VALUES (@Username, @Password)";
- "INSERT INTO customerinfo VALUES (@CustomerID, @CustomerName, @Gender, @Email, @Phone, @Username)";
- "UPDATE customerinfo SET customername=@CustomerName, gender=@Gender, email=@Email, phone=@Phone WHERE customerid=@CustomerID";
- "SELECT Username FROM customerinfo WHERE customerid=@CustomerID";
- "DELETE FROM customerinfo WHERE customerid=@CustomerID";
- "DELETE FROM UserInfo WHERE Username=@Username";
- "SELECT ISNULL(MAX(customerid),0)+1 FROM customerinfo";

Car:

- SELECT * FROM cardetails;
- "INSERT INTO cardetails VALUES (@CarID, @CarName, @CarNumber, @CarModel, @RentPrice, @CarStatus)";
- "UPDATE cardetails SET carname=@CarName, carnumber=@CarNumber, carmodel=@CarModel, rentprice=@RentPrice, carstatus=@CarStatus WHERE carid=@CarID";
- "SELECT ISNULL(MAX(carid),0)+1 FROM cardetails";

Rental:

- "INSERT INTO Rental VALUES (@RentID, @CustomerName, @CarName, @CarModel, @StartDate, @EndDate, @RentalStatus)";
- "UPDATE Rental SET customername=@CustomerName, carname=@CarName, carmodel=@CarModel, startdate=@StartDate, enddate=@EndDate, rentalstatus=@RentalStatus WHERE rentid=@RentID";
- "DELETE FROM Rental WHERE rentid=@RentID";
- "UPDATE Rental SET rentalstatus='Active' WHERE rentid=@RentID";
- "UPDATE cardetails SET carstatus='Rented' WHERE carname=@CarName AND carmodel=@CarModel";

Driver:

- sELECT * FROM driver;
- "INSERT INTO driver VALUES (@DriverID, @DriverName, @Gender, @Phone)";
- "UPDATE driver SET drivername=@DriverName, drivergender=@Gender, phone=@Phone WHERE driverid=@DriverID";
- "DELETE FROM driver WHERE driverid=@DriverID";
- "SELECT ISNULL(MAX(driverid),0)+1 FROM driver";

Maintenance:

- SELECT * FROM maintenance;
- "INSERT INTO maintenance VALUES (@MaintenanceID, @CarName, @CarModel, @Service, @Cost)";
- "UPDATE maintenance SET carname=@CarName, carmodel=@CarModel, service=@Service, cost=@Cost WHERE maintenanceid=@MaintenanceID";
- "DELETE FROM maintenance WHERE maintenanceid=@MaintenanceID";
- "SELECT ISNULL(MAX(maintenanceid),0)+1 FROM maintenance";

Rent Car:

- "SELECT * FROM cardetails WHERE carname=@CarName AND carmodel=@CarModel AND carstatus='Available'";
- "SELECT customername FROM customerinfo WHERE Username=@Username";
- "SELECT ISNULL(MAX(rentid),0)+1 AS nextId FROM Rental";
- "INSERT INTO Rental VALUES (@RentID, @CustomerName, @CarName, @CarModel, @StartDate, @EndDate, 'Pending')";
- "UPDATE cardetails SET carstatus='Reserved' WHERE carname=@CarName AND carmodel=@CarModel";
- "SELECT * FROM Rental WHERE customername=@CustomerName";

View Car:

- "SELECT * FROM cardetails WHERE carstatus='Available' AND (carname LIKE @Search OR carmodel LIKE @Search)";

Chapter 04

Screenshot:

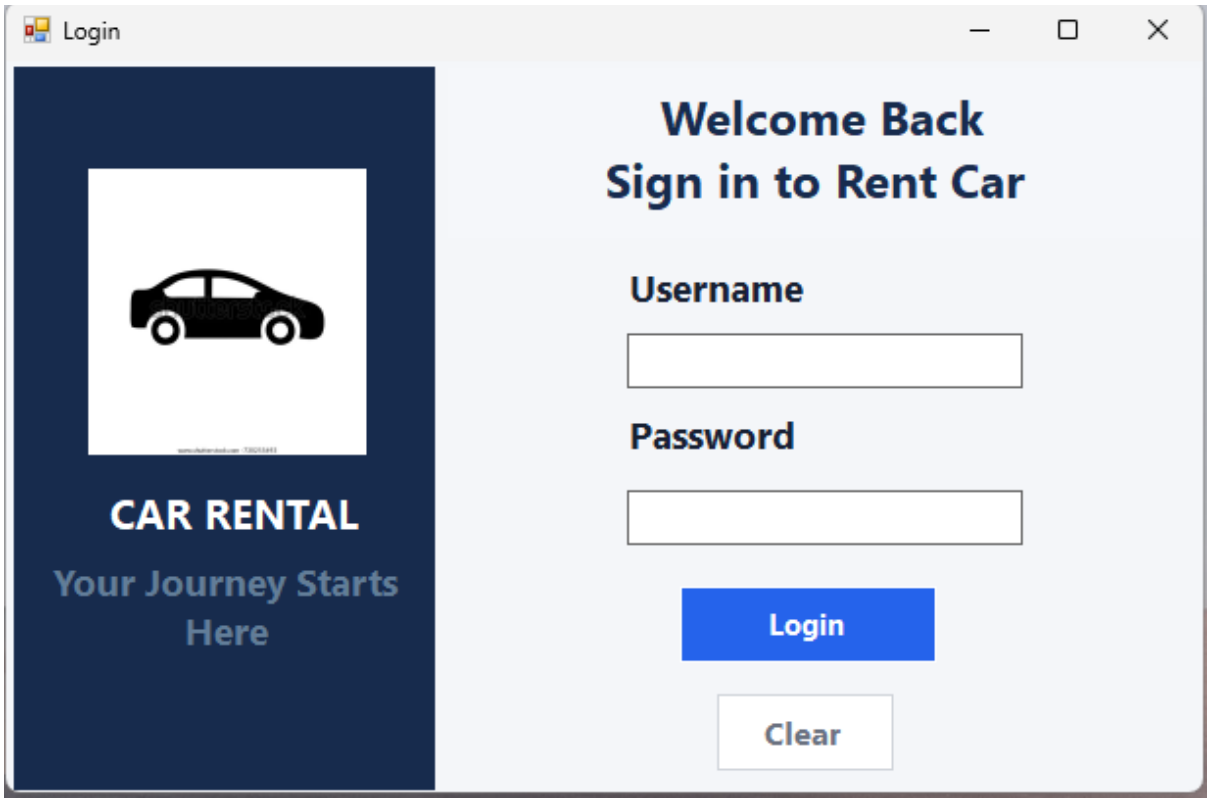


Figure 1


Main
Car Rental Management
Dashboard
Customer
Car
Rental
Driver
Maintenance
Logout
43
Total Customers
25
Available Cars
13
Active Rentals
30
Drivers

Figure 2

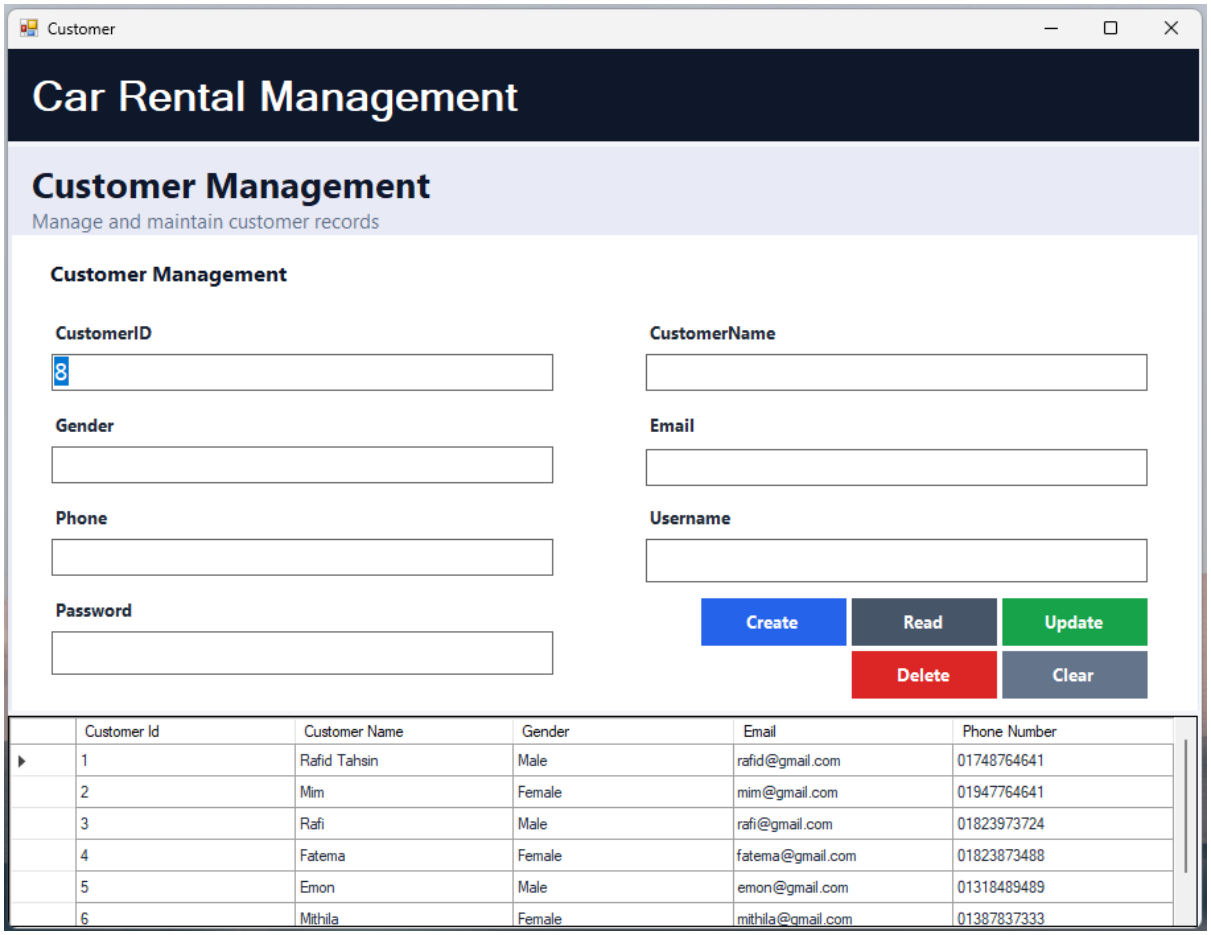

Customer
Car Rental Management
Customer Management
Manage and maintain customer records
Customer Management
CustomerID
8
CustomerName
Gender
Email
Phone
Username
Password
Create
Read
Update
Delete
Clear
Customer Id
Customer Name
Gender
Email
Phone Number
1
Rafid Tahsin
Male
rafid@gmail.com
01748764641
2
Mim
Female
mim@gmail.com
01947764641
3
Rafi
Male
rafi@gmail.com
01823973724
4
Fatema
Female
fatema@gmail.com
01823873488
5
Emon
Male
emon@gmail.com
01318489489
6
Mithila
Female
mithila@gmail.com
01387837333

Figure 3

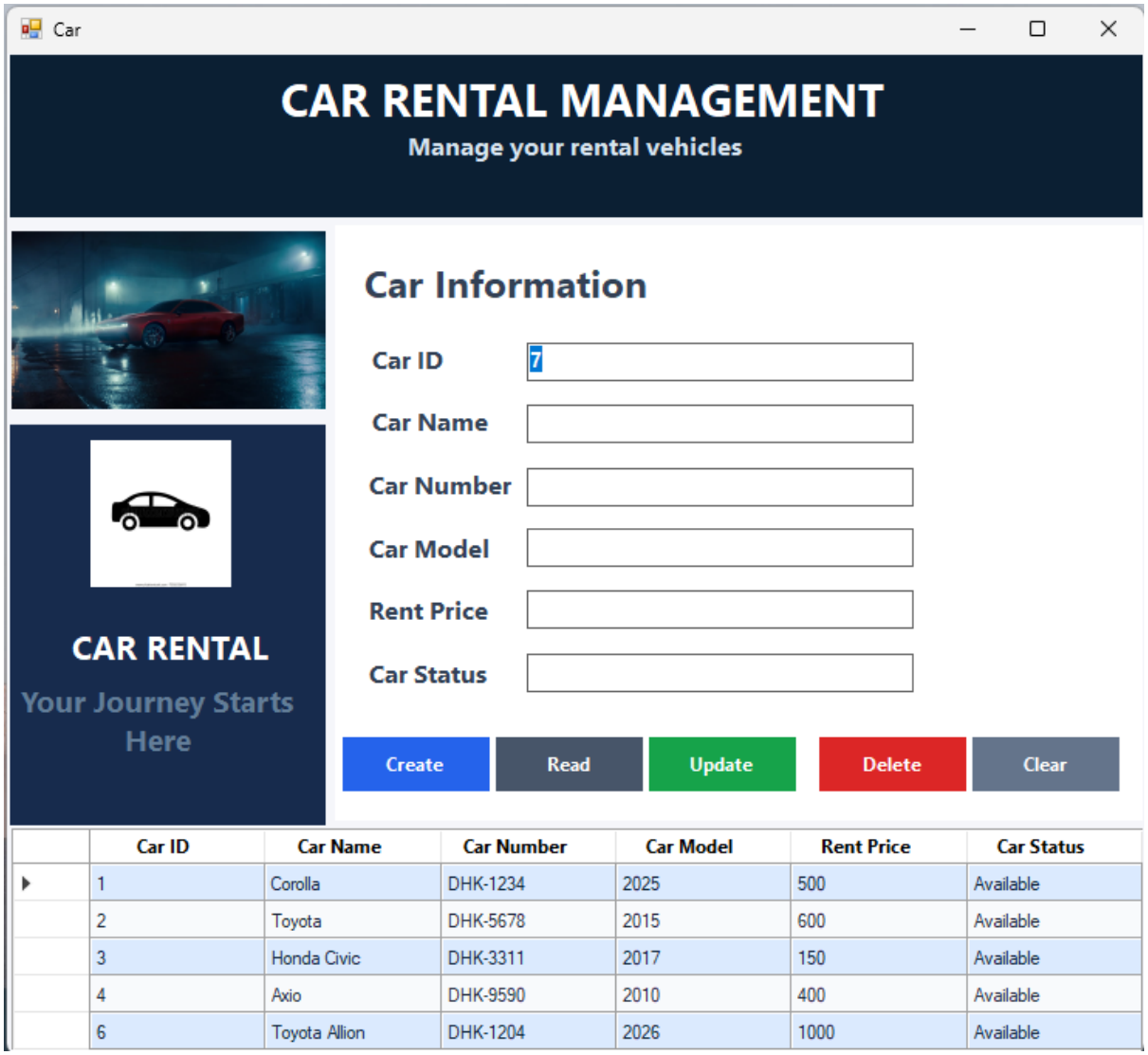

Car
CAR RENTAL MANAGEMENT
Manage your rental vehicles
Car Information
Car ID
7
Car Name
Car Number
Car Model
Rent Price
Car Status
CAR RENTAL
Your Journey Starts Here
Create
Read
Update
Delete
Clear
Car ID
Car Name
Car Number
Car Model
Rent Price
Car Status
1
Corolla
DHK-1234
2025
500
Available
2
Toyota
DHK-5678
2015
600
Available
3
Honda Civic
DHK-3311
2017
150
Available
4
Axio
DHK-9590
2010
400
Available
6
Toyota Allion
DHK-1204
2026
1000
Available

Figure 4

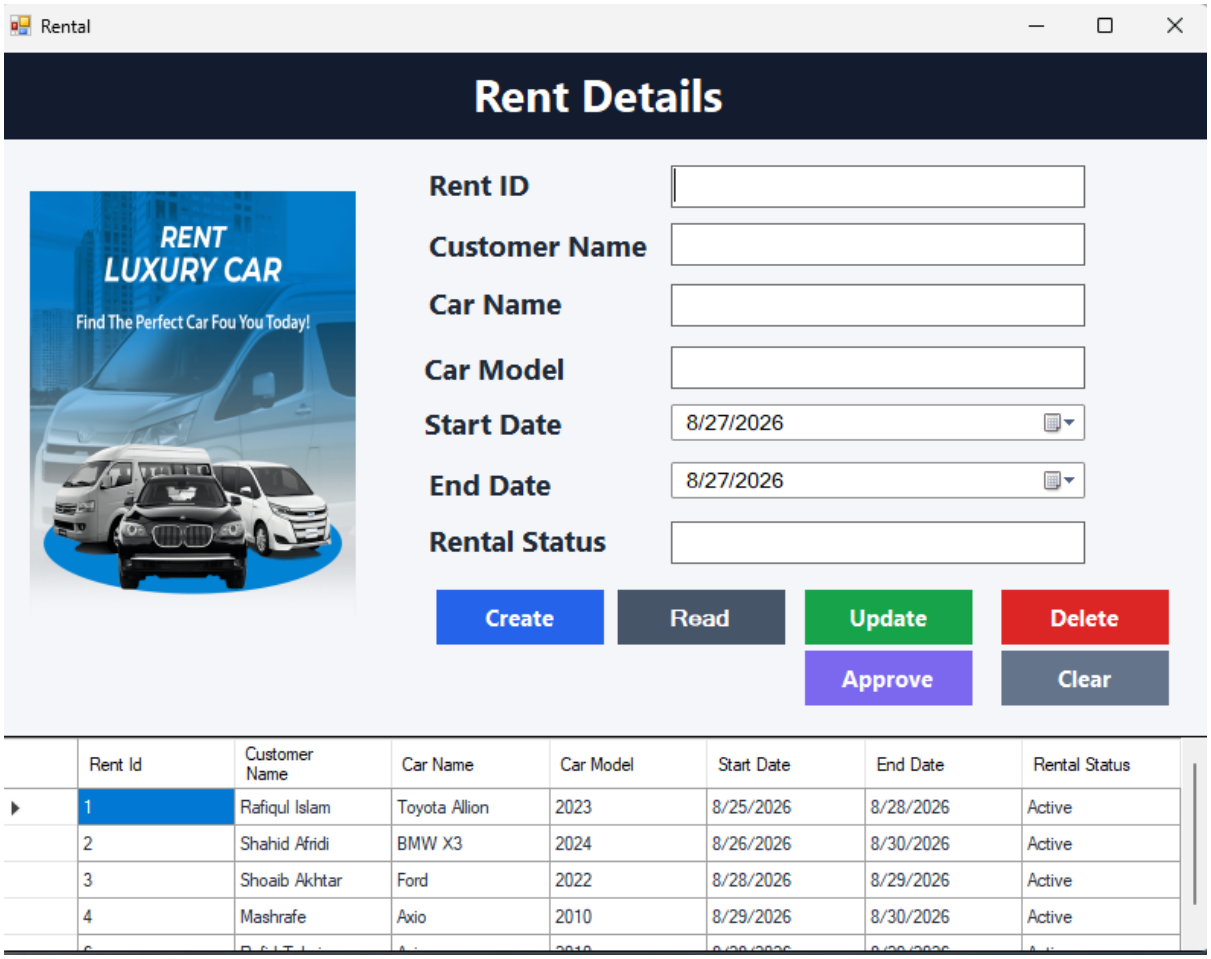

Rental
Rent Details
RENT
LUXURY CAR
Find The Perfect Car Fou You Today!
Rent ID
Customer Name
Car Name
Car Model
Start Date
8/27/2026
End Date
8/27/2026
Rental Status
Create
Read
Update
Delete
Approve
Clear
Rent Id
Customer Name
Car Name
Car Model
Start Date
End Date
Rental Status
1
Rafiqul Islam
Toyota Allion
2023
8/25/2026
8/28/2026
Active
2
Shahid Afridi
BMW X3
2024
8/26/2026
8/30/2026
Active
3
Shoaib Akhtar
Ford
2022
8/28/2026
8/29/2026
Active
4
Mashrafe
Axio
2010
8/29/2026
8/30/2026
Active

Figure 5

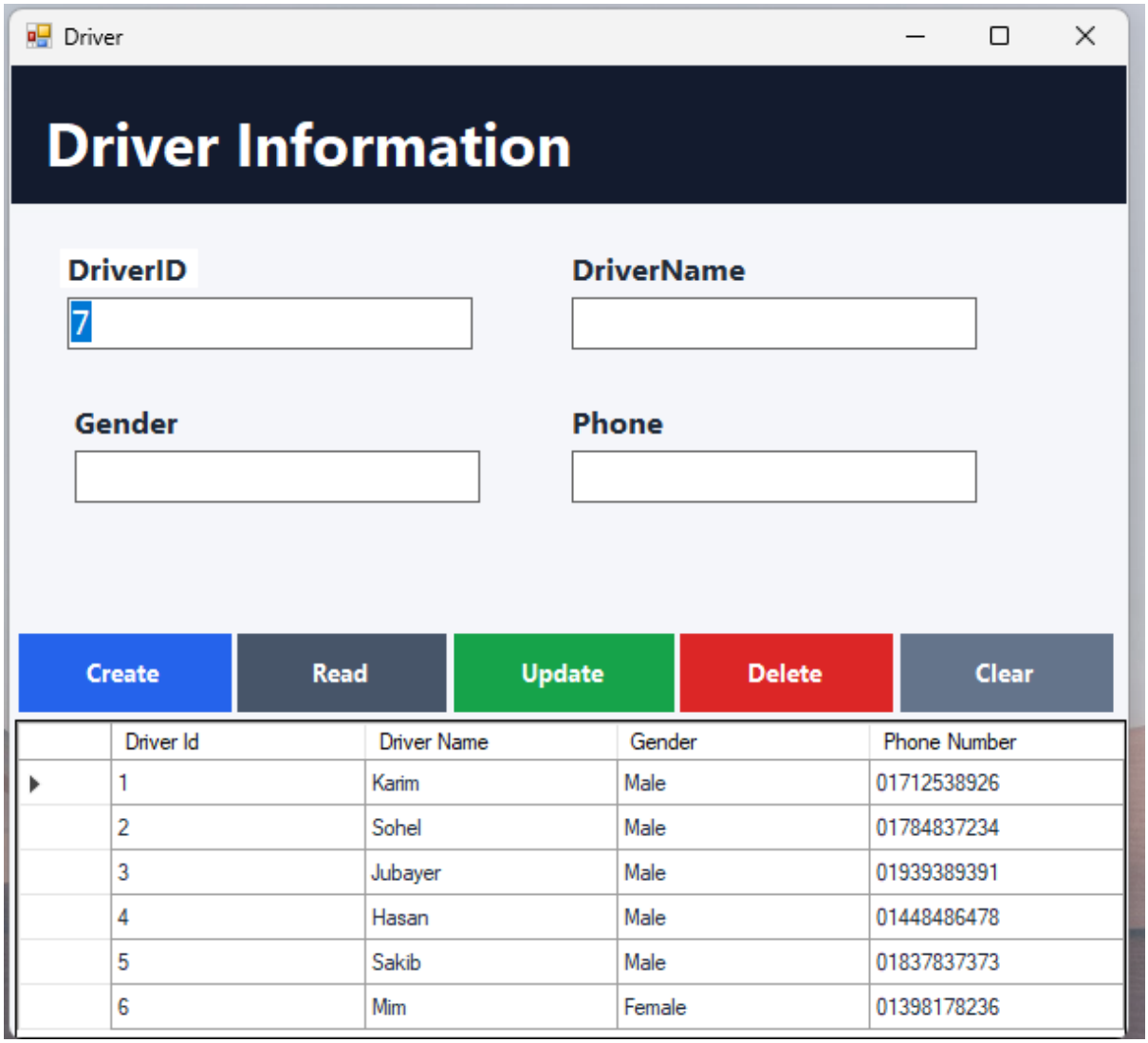

Driver
Driver Information
DriverID
7
DriverName
Gender
Phone
Create
Read
Update
Delete
Clear
Driver Id
Driver Name
Gender
Phone Number
1
Karim
Male
01712538926
2
Sohel
Male
01784837234
3
Jubayer
Male
01939389391
4
Hasan
Male
01448486478
5
Sakib
Male
01837837373
6
Mim
Female
01398178236

Figure 6



Figure 7

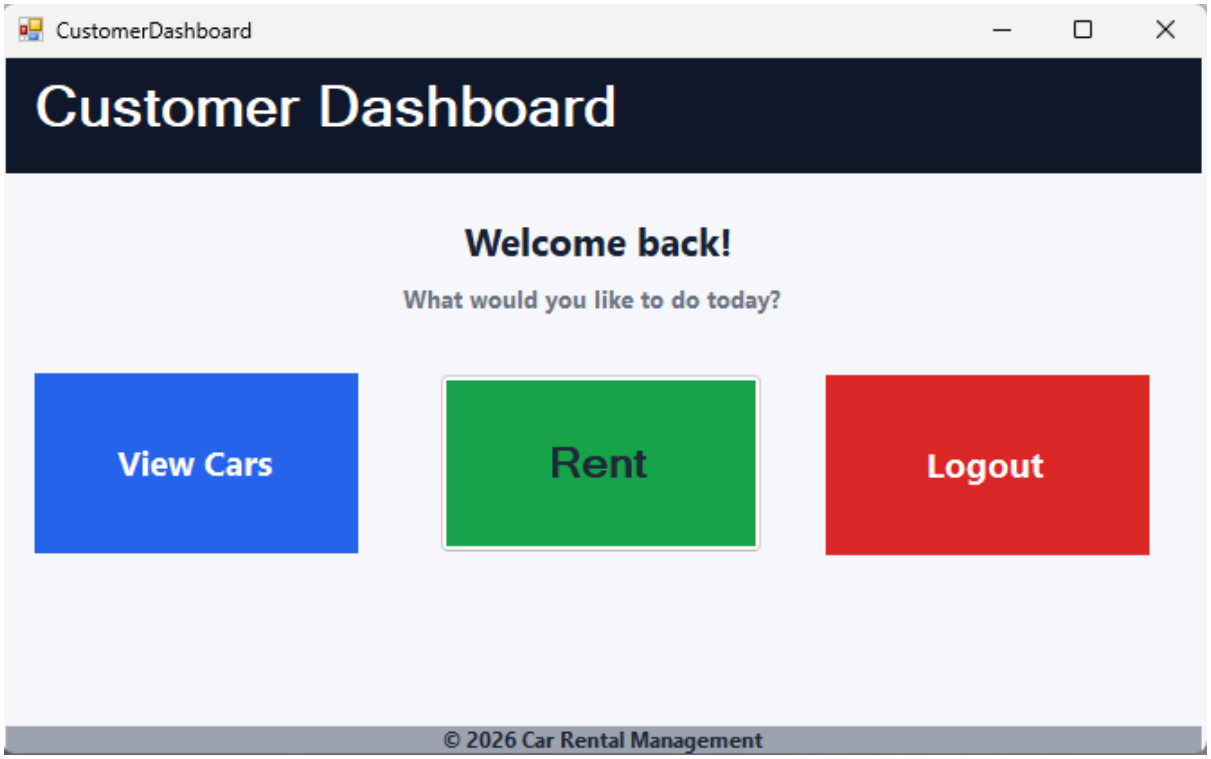
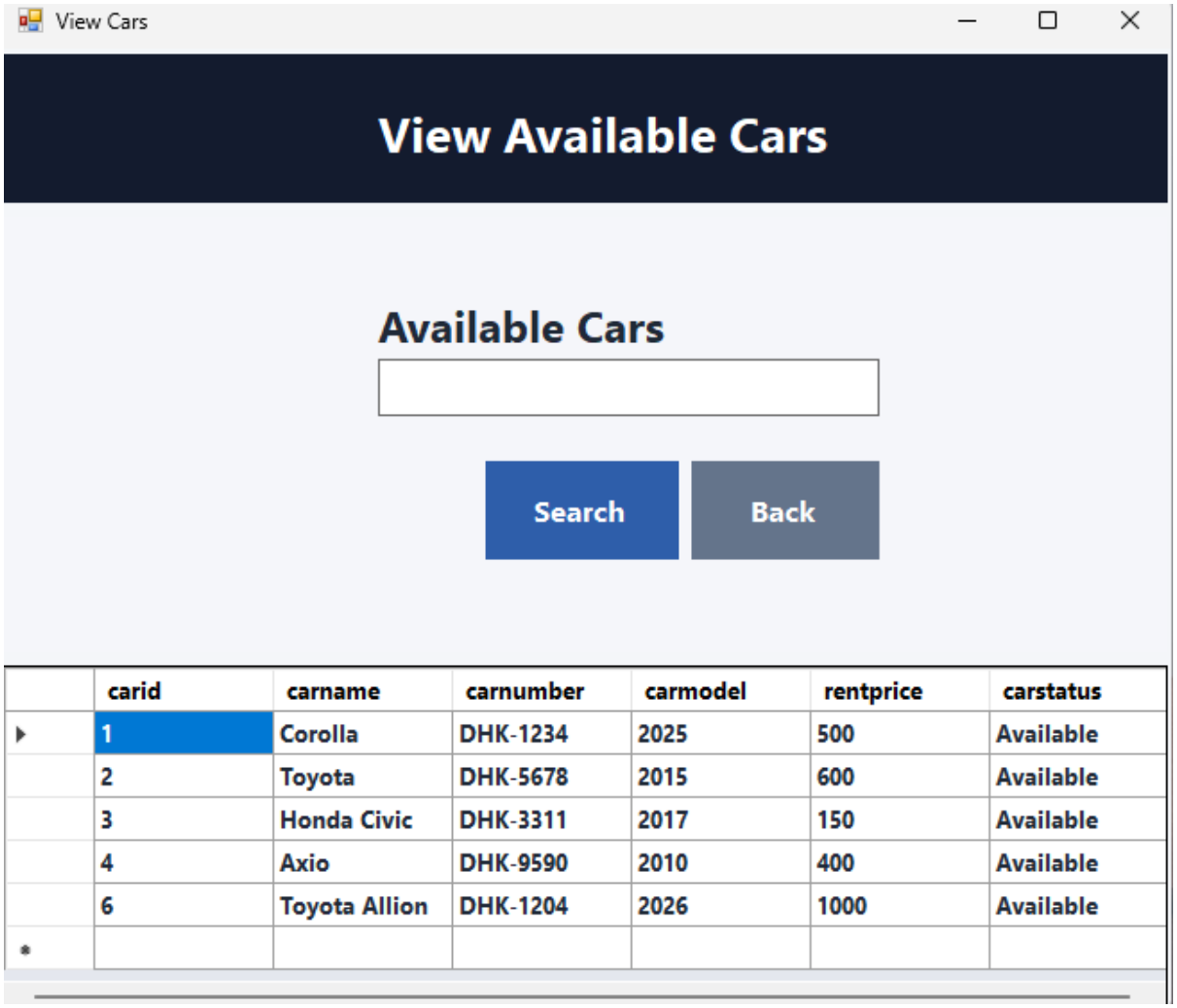


Figure 8



	carid	carname	carnumber	carmodel	rentprice	carstatus
▶	1	Corolla	DHK-1234	2025	500	Available
	2	Toyota	DHK-5678	2015	600	Available
	3	Honda Civic	DHK-3311	2017	150	Available
	4	Axio	DHK-9590	2010	400	Available
	6	Toyota Allion	DHK-1204	2026	1000	Available
*						

Figure 9

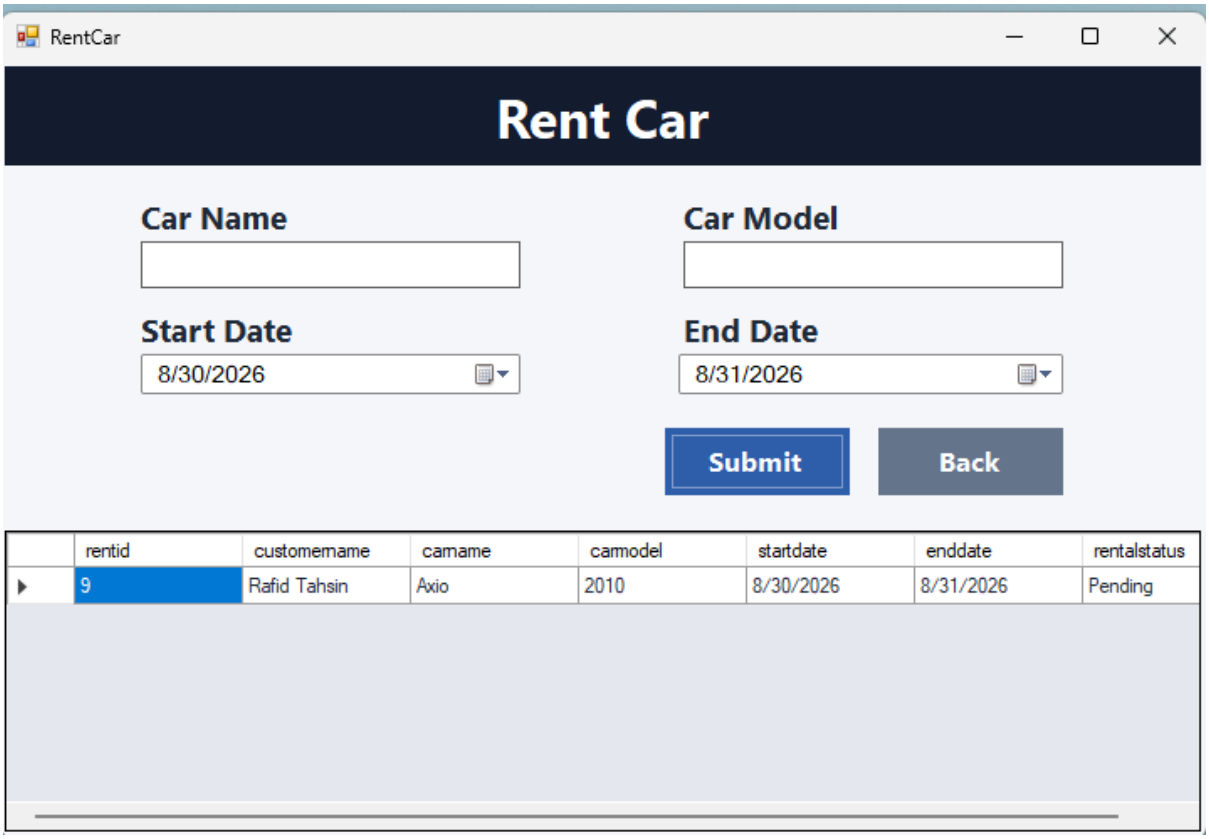

RentCar
Rent Car
Car Name
Car Model
Start Date
8/30/2026
End Date
8/31/2026
Submit
Back
rentid
customemame
camame
carmodel
startdate
enddate
rentalstatus
9
Rafid Tahsin
Axio
2010
8/30/2026
8/31/2026
Pending

Figure 10

Chapter 05 : Code :

Login Code:

```
namespace Car_Rent
{
   public partial class Login : Form
   {
      public DataAccess Da { get; set; }

      public Login()
      {
         InitializeComponent();
         this.Da = new DataAccess();
      }

      private void btnLogin_Click(object sender, EventArgs e)
      {
         string username = txtUsername.Text;
         string password = txtPassword.Text;

         DataTable adminDt = this.Da.ExecuteQueryTable("select AdminUsername,AdminPassword
from AdminInfo where AdminUsername = '" + username + "'and AdminPassword ='" + password +
"'");

         if (adminDt.Rows.Count > 0)
         {
            Main mn = new Main();
            mn.Show();
            this.Hide();
            return;
         }

         DataTable custDt = this.Da.ExecuteQueryTable("select Username,Password from UserInfo
where Username = '" + username + "'and Password ='" + password + "'");

         if (custDt.Rows.Count > 0)
         {
            CustomerDashboard cd = new CustomerDashboard(username);
            cd.Show();
            this.Hide();
         }
         else
         {
            MessageBox.Show("Invalid Login!!");
         }
      }

      private void Login_Load(object sender, EventArgs e){     }
       private void btnClear_Click(object sender, EventArgs e)
      {
```

```
            txtUsername.Text = "";
            txtPassword.Text = "";
        }

    }
}
```

Customer Code:

```
namespace Car_Rent
{
    public partial class Customer : Form
    {
        public DataAccess Da { get; set; }

        public Customer()
        {
            InitializeComponent();
            this.Da = new DataAccess();

            this.PopulateGridView();
            this.AutoIdGenerate();
        }

        private void PopulateGridView(string sql = "select * from customerinfo;")
        {
            var ds = this.Da.ExecuteQuery(sql);

            this.dgvCustomerInfo.AutoGenerateColumns = false;
            this.dgvCustomerInfo.DataSource = ds.Tables[0];
        }

        private void Customer_Load(object sender, EventArgs e)
        {
            this.dgvCustomerInfo.ClearSelection();
            this.AutoIdGenerate();
        }

        private void btnCreate_Click(object sender, EventArgs e)
        {
            try
            {
                if (!this.IsValidToCreate())
                {
                    MessageBox.Show("Please fill all the empty fields");
                    return;
                }

                var userSql = "insert into UserInfo values('" + this.txtUsername.Text + "', '" +
this.txtPassword.Text + "');";
```

```
            this.Da.ExecuteDMLQuery(userSql);

            var customerSql = "insert into customerinfo values('" + this.txtCustomerID.Text + "', '" +
this.txtCustomerName.Text + "', '" + this.txtGender.Text + "', '" + this.txtEmail.Text + "', '" +
this.txtPhone.Text + "', '" + this.txtUsername.Text + "');";
            var count = this.Da.ExecuteDMLQuery(customerSql);

            if (count == 1)
               MessageBox.Show("Created Successfully");
            else
               MessageBox.Show("Data hasn't been added to the list");

            this.PopulateGridView();
            this.ClearAll();
         }
         catch (Exception exc)
         {
            MessageBox.Show("An error has occurred in the system, please try again.\n" +
exc.Message);
         }
      }

      private bool IsValidToCreate()
      {
         if (string.IsNullOrEmpty(this.txtCustomerID.Text) ||
string.IsNullOrEmpty(this.txtUsername.Text) || string.IsNullOrEmpty(this.txtPassword.Text) ||
string.IsNullOrEmpty(this.txtCustomerName.Text) || string.IsNullOrEmpty(this.txtGender.Text) ||
string.IsNullOrEmpty(this.txtEmail.Text) || string.IsNullOrEmpty(this.txtPhone.Text))
            return false;
         else
            return true;
      }

      private void btnRead_Click(object sender, EventArgs e)
      {
         this.PopulateGridView();
      }

      private void btnUpdate_Click(object sender, EventArgs e)
      {
         try
         {
            if (!this.IsValidToUpdate())
            {
               MessageBox.Show("Please fill all the empty fields");
               return;
            }

            var sql = @"update customerinfo
            set customername = '" + this.txtCustomerName.Text + @"',
```

```
            gender = '" + this.txtGender.Text + @"',
            email = '" + this.txtEmail.Text + @"',
            phone = '" + this.txtPhone.Text + @"'
            where customerid = '" + this.txtCustomerID.Text + "';";

            var count = this.Da.ExecuteDMLQuery(sql);

            if (count == 1)
               MessageBox.Show("Updated Successfully");
            else
               MessageBox.Show("Data hasn't been updated in the list");

            this.PopulateGridView();
            this.ClearAll();
         }
         catch (Exception exc)
         {
            MessageBox.Show("An error has occurred in the system, please try again.\n" +
exc.Message);
         }
      }

      private bool IsValidToUpdate()
      {
         if (string.IsNullOrEmpty(this.txtCustomerID.Text) ||
string.IsNullOrEmpty(this.txtCustomerName.Text) || string.IsNullOrEmpty(this.txtGender.Text) ||
string.IsNullOrEmpty(this.txtEmail.Text) || string.IsNullOrEmpty(this.txtPhone.Text))
            return false;
         else
            return true;
      }

      private void btnDelete_Click(object sender, EventArgs e)
      {
         try
         {
            if (this.dgvCustomerInfo.SelectedRows.Count < 1)
            {
               MessageBox.Show("Please select a row first to delete.", "Alert",
MessageBoxButtons.OK, MessageBoxIcon.Stop);
               return;
            }

            var CustomerName =
this.dgvCustomerInfo.CurrentRow.Cells["dgvCustomerName"].Value.ToString();
            var CustomerID =
this.dgvCustomerInfo.CurrentRow.Cells["dgvCustomerID"].Value.ToString();

            var usernameSql = "select Username from customerinfo where customerid = '" +
CustomerID + "';";
```

```
            DataTable dt = this.Da.ExecuteQueryTable(usernameSql);
            var Username = dt.Rows[0]["Username"].ToString();

            var result = MessageBox.Show("Are you sure to delete data?", "Warning",
MessageBoxButtons.YesNo, MessageBoxIcon.Exclamation);
            if (result == DialogResult.No)
               return;

            var sql = "delete from customerinfo where customerid = '" + CustomerID + "';";
            var count = this.Da.ExecuteDMLQuery(sql);

            if (count == 1)
            {
               var userSql = "delete from UserInfo where Username = '" + Username + "';";
               this.Da.ExecuteDMLQuery(userSql);
               MessageBox.Show(CustomerName.ToUpper() + " has been removed from the list");
            }
            else
            {
               MessageBox.Show("Data hasn't been deleted from the list");
            }

            this.PopulateGridView();
            this.ClearAll();
         }
         catch (Exception exc)
         {
            MessageBox.Show("An error has occurred in the system, please try again.\n" +
exc.Message);
         }
      }

      private void btnClear_Click(object sender, EventArgs e)
      {
         this.ClearAll();
      }

      private void AutoIdGenerate()
      {
         var sql = "select isnull(max(customerid),0)+1 from customerinfo;";
         DataTable dt = this.Da.ExecuteQueryTable(sql);
         var nextId = dt.Rows[0][0].ToString();
         this.txtCustomerID.Text = nextId;
      }

      private void dgvCustomerInfo_DoubleClick(object sender, EventArgs e)
      {
         this.txtCustomerID.Text =
this.dgvCustomerInfo.CurrentRow.Cells["dgvCustomerID"].Value.ToString();
         this.txtCustomerName.Text =
```

```
this.dgvCustomerInfo.CurrentRow.Cells["dgvCustomerName"].Value.ToString();
            this.txtGender.Text =
this.dgvCustomerInfo.CurrentRow.Cells["dgvGender"].Value.ToString();
            this.txtEmail.Text = this.dgvCustomerInfo.CurrentRow.Cells["dgvEmail"].Value.ToString();
            this.txtPhone.Text = this.dgvCustomerInfo.CurrentRow.Cells["dgvPhone"].Value.ToString();
        }

        private void ClearAll()
        {
            this.txtUsername.Clear();
            this.txtPassword.Clear();
            this.txtCustomerName.Clear();
            this.txtGender.Clear();
            this.txtEmail.Clear();
            this.txtPhone.Clear();

            this.dgvCustomerInfo.ClearSelection();
            this.AutoIdGenerate();
        }
    }
}
```

Data Access Code:

```
namespace Car_Rent
{
    public class DataAccess
    {
        private SqlConnection sqlcon;
        public SqlConnection Sqlcon
        {
            get { return this.sqlcon; }
            set { this.sqlcon = value; }
        }

        private SqlCommand sqlcom;
        public SqlCommand Sqlcom
        {
            get { return this.sqlcom; }
            set { this.sqlcom = value; }
        }

        private SqlDataAdapter sda;
        public SqlDataAdapter Sda
        {
            get { return this.sda; }
            set { this.sda = value; }
        }
```

```
        private DataSet ds;
        public DataSet Ds
        {
            get { return this.ds; }
            set { this.ds = value; }
        }

        public DataAccess()
        {
            this.Sqlcon = new SqlConnection(@"Data Source=DESKTOP-
8LUJLHK\SQLEXPRESS;Initial Catalog=CarRentDB;Integrated Security=True;");
            Sqlcon.Open();
        }

        private void QueryText(string query)
        {
            this.Sqlcom = new SqlCommand(query, this.Sqlcon);
        }

        public DataSet ExecuteQuery(string sql)
        {
            this.Sqlcom = new SqlCommand(sql, this.Sqlcon);
            this.Sda = new SqlDataAdapter(this.Sqlcom);
            this.Ds = new DataSet();
            this.Sda.Fill(this.Ds);
            return Ds;
        }

        public DataTable ExecuteQueryTable(string sql)
        {
            this.Sqlcom = new SqlCommand(sql, this.Sqlcon);
            this.Sda = new SqlDataAdapter(this.Sqlcom);
            this.Ds = new DataSet();
            this.Sda.Fill(this.Ds);
            return Ds.Tables[0];
        }

        public int ExecuteDMLQuery(string sql)
        {
            this.Sqlcom = new SqlCommand(sql, this.Sqlcon);
            int u = this.Sqlcom.ExecuteNonQuery();
            return u;
        }
    }
}
```